

Master Unit Testing & Mockito Technical Guide

Exhaustive Code-Level Walkthrough across Auth, Property, Booking, and Payment Services

Stack: JUnit 5 (Jupiter) | Mockito 5 | MockMvc | Jakarta Bean Validation | **Test Status:** 103 Tests Executed, 0 Failures, 0 Errors **Scope:** Complete Microservice Fleet

1. THE FUNDAMENTALS OF UNIT TESTING & MOCKITO MECHANICS

Unit testing verifies that a small, isolated piece of code (a single method or class) performs exactly as intended under both valid and erroneous inputs. In enterprise Spring Boot development, **unit tests do not start an actual web server (Tomcat) and do not connect to a physical MySQL database.**

Concept / Annotation	What it Does in Code	Why We Use It (Enterprise Benefit)
@ExtendWith(MockitoExtension.class)	Initializes the Mockito testing framework on JUnit 5 test classes.	Enables automatic initialization of mocks without writing boilerplate openMocks() setup code.
@Mock	Creates a fake dummy instance of a dependency (e.g., UserRepository).	Isolates the class under test. No real database queries or network sockets are ever touched.
@InjectMocks	Creates the real instance of the service and injects all @Mock fields into it.	Emulates Spring's Dependency Injection (DI) mechanism inside memory in a fraction of a millisecond.
when(...).thenReturn(...)	Stubbing: Instructs a mock object to return a predefined value when called.	Simulates any scenario (e.g., user exists, property found, credit score high) on demand.
assertThrows(...)	Verifies that executing a lambda throws the exact expected exception.	Essential for negative testing (verifying that invalid inputs trigger proper 400/404/409 errors).
verify(mock, times(1))	Checks whether a mock method was invoked the expected number of times.	Proves that side-effects happened (e.g., save() called once) or didn't (never() called on errors).
MockMvc	Spring's virtual HTTP client that executes controller endpoints in-memory.	Tests HTTP routing, JSON serialization, and GlobalExceptionHandler without starting a real port.

💡 The AAA (Arrange-Act-Assert) Pattern:

Every test method in our project follows this clean structure: **1. Arrange (Given):** Prepare mock inputs and stub mock behaviors with when().thenReturn().

2. Act (When): Invoke the actual service method (e.g., authService.register(request)).

3. Assert (Then): Check returned values with assertEquals / assertNotNull, and verify interactions with verify().

2. AUTH SERVICE TESTING (57 TESTS – 100% PASS)

Test Suite: AuthServiceTest.java (15 Tests), RegisterRequestValidationTest.java (33 Tests),
AuthControllerValidationTest.java (5 Tests), JwtUtilTest.java (4 Tests).

A. Service Layer: Happy Path & Duplicate User Defense (`AuthServiceTest.java`)

```
// 1. SUCCESSFUL REGISTRATION TEST
@Test
void testRegister_Success() {
    RegisterRequest request = new RegisterRequest();
    request.setUsername("newuser");
    request.setEmail("new@example.com");
    request.setPassword("password123");
    request.setFullName("New User");
    request.setPhoneNumber("9999999999");
    request.setRole("TENANT");

    // Stubbing repository checks
    when(userRepository.existsByUsername("newuser")).thenReturn(false);
    when(userRepository.existsByEmail("new@example.com")).thenReturn(false);
    when(passwordEncoder.encode("password123")).thenReturn("hashed123");
    when(userRepository.save(any(User.class))).thenAnswer(inv -> {
```

```

        User u = inv.getArgument(0);
        u.setId(10L);
        return u;
    });

    UserResponse response = authService.register(request);

    assertNotNull(response);
    assertEquals(10L, response.getId());
    assertEquals("newuser", response.getUsername());
    assertEquals("ROLE_TENANT", response.getRole());
    verify(userRepository, times(1)).save(any(User.class)); // Database save verified!
}

// 2. DUPLICATE USERNAME NEGATIVE TEST
@Test
void testRegister_DuplicateUsername() {
    RegisterRequest request = new RegisterRequest();
    request.setUsername("testuser");

    when(userRepository.existsByUsername("testuser")).thenReturn(true);

    assertThrows(DuplicateResourceException.class, () -> authService.register(request));
    verify(userRepository, never()).save(any(User.class)); // Ensures DB is NOT corrupted!
}

```

B. Bean Validation Constraints (`RegisterRequestValidationTest.java`)

Directly tests Jakarta annotations (`@NotBlank`, `@Pattern`, `@Size`) against boundary edge cases:

- **Full Name Regex** (`^[a-zA-Z\s]{2,50}\$`): Parametric tests reject digits and symbols (e.g. "HARSH SAHU1", "User123", "Name!") with 100% assertion coverage.
- **Indian Phone Numbers** (`^[6-9]\d{9}\$`): Rejects 9-digit phones ("987654321"), 11-digit phones, alphabetic strings, and invalid starting digits ("5876543210").

C. HTTP MockMvc Endpoints (`AuthControllerValidationTest.java`)

```

@Test
void testRegister_InvalidEmail_Returns400() throws Exception {
    RegisterRequest request = createValidRequest();
    request.setEmail("@abc.com"); // Malformed email

    mockMvc.perform(post("/api/auth/register")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(request)))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.status").value(400))
        .andExpect(jsonPath("$.message").value("Validation failed"))
        .andExpect(jsonPath("$.details[0]").value(containsString("email")));

    verify(userRepository, never()).save(any(User.class));
}

```

3. PROPERTY SERVICE TESTING (18 TESTS – 100% PASS)

Test Suite: `PropertyServiceTest.java`

Core Areas Tested: CRUD operations, Ownership Authorization, Caching, and the **AI Rent Estimation Algorithm**.

A. AI Rent Estimation Model Tests

Tests the mathematical heuristic algorithm in `PropertyService.java` verifying that city base rates, bedroom layouts, furnishing multipliers, and amenities compute an accurate market valuation with bounds and confidence intervals:

```
@Test
@DisplayName("Should accurately estimate market rent for Bangalore 2BHK furnished flat")
void testEstimateRent_Bangalore_Success() {
    RentEstimationRequest request = new RentEstimationRequest();
    request.setCity("Bangalore");
    request.setLocality("Indiranagar");
    request.setArea(1000);
    request.setBedrooms(2);
    request.setFurnishing("Furnished");
    request.setHasParking(true);
    request.setHasGym(true);

    RentEstimationResponse response = propertyService.estimateRent(request);

    assertNotNull(response);
    assertTrue(response.getEstimatedRent().compareTo(BigDecimal.ZERO) > 0);
    assertTrue(response.getConfidenceScore() >= 90); // High confidence score assertion
    assertNotNull(response.getSummary());
    assertTrue(response.getSummary().contains("Bangalore"));

    // Bounds verification: MinRent < EstimatedRent < MaxRent
    assertTrue(response.getMinRent().compareTo(response.getEstimatedRent()) < 0);
    assertTrue(response.getMaxRent().compareTo(response.getEstimatedRent()) > 0);
}

@Test
@DisplayName("Should throw IllegalArgumentException when rent estimation request is null")
void testEstimateRent_NullRequest_ThrowsException() {
    assertThrows(IllegalArgumentException.class, () -> propertyService.estimateRent(null));
}
```

B. Listing Ownership Security Test

Prevents Insecure Direct Object References (IDOR). If an attacker attempts to update or delete a listing owned by another landlord, the service rejects the call:

```
@Test
void testUpdateProperty_UnauthorizedOwner_ThrowsException() {
    when(propertyRepository.findById(1L)).thenReturn(Optional.of(sampleProperty)); // ownerId = 10L

    PropertyRequest updateReq = new PropertyRequest();
    updateReq.setTitle("Hacked Title");

    // Calling with requestingUserId = 999L (attacker)
    assertThrows(UnauthorizedActionException.class,
        () -> propertyService.updateProperty(1L, updateReq, 999L));

    verify(propertyRepository, never()).save(any(Property.class));
}
```

4. BOOKING SERVICE TESTING (15 TESTS – 100% PASS)

Test Suite: `BookingServiceTest.java`

Core Areas Tested: Date validation, double-booking rejection, state machine transitions, and **Automated Tenant KYC & CIBIL Verification**.

A. Tenant Government KYC Verification Engine

Validates Indian UIDAI Aadhaar (12 digits) and Income Tax PAN (10 chars), tests automated data masking for privacy (CWE/DPDP compliance), and validates calculated CIBIL credit scores:

```

@Test
@DisplayName("Should successfully verify tenant KYC with valid Aadhaar, PAN and compute high CIBIL score")
void testVerifyTenantKyc_Success() {
    TenantKycRequest kycReq = new TenantKycRequest();
    kycReq.setFullName("Aman Verma");
    kycReq.setAadhaarNumber("987654321098");
    kycReq.setPanNumber("ABCDE1234F");
    kycReq.setCompanyName("Infosys Limited");
    kycReq.setMonthlyIncome(85000.0);

    TenantKycResponse response = bookingService.verifyTenantKyc(200L, kycReq, 200L);

    assertNotNull(response);
    assertEquals("VERIFIED", response.getKycStatus());
    assertEquals("LOW", response.getRiskLevel());
    assertTrue(response.getCibilScore() >= 750); // CIBIL score assertion
    assertEquals("XXXX-XXXX-1098", response.getAadhaarMasked()); // Privacy masking check
    assertEquals("AB*****4F", response.getPanMasked());
    assertNotNull(response.getReferenceNumber());
}

@Test
@DisplayName("Should reject KYC verification when requesting user does not match tenant ID")
void testVerifyTenantKyc_WrongUser_ThrowsException() {
    TenantKycRequest kycReq = new TenantKycRequest();
    kycReq.setAadhaarNumber("987654321098");
    kycReq.setPanNumber("ABCDE1234F");

    // Requester 999L is not tenant 200L
    assertThrows(InvalidBookingException.class,
        () -> bookingService.verifyTenantKyc(200L, kycReq, 999L));
}

```

B. Date Conflict & Overlap Detection Test

Tests chronological sanity (end date after start date) and rejects missing or null dates:

```

@Test
void testCreateBooking_NullStartDate_ThrowsException() {
    BookingRequest request = new BookingRequest();
    request.setPropertyId(100L);
    request.setStartDate(null);
    request.setEndDate(LocalDate.of(2026, 9, 30));

    assertThrows(InvalidBookingException.class, () -> bookingService.createBooking(request, 200L));
    verify(bookingRepository, never()).save(any(Booking.class));
}

```

5. PAYMENT SERVICE TESTING (13 TESTS – 100% PASS)

Test Suite: `MockPaymentServiceTest.java`

Core Areas Tested: Payment creation, high-entropy transaction ID generation, idempotency (duplicate payment defense), financial boundary guards (zero/negative amounts), and refund authorization.

A. Financial Boundary & Zero/Negative Amount Defense

Financial services must rigorously reject zero or negative payments to prevent ledger corruption:

```
@Test
@DisplayName("Should reject payment with zero amount")
void testCreatePayment_ZeroAmount_ThrowsException() {
    PaymentRequest request = new PaymentRequest();
    request.setBookingId(100L);
    request.setAmount(BigDecimal.ZERO); // ₹0.00
    request.setPaymentMethod(PaymentMethod.CARD);
    request.setPaymentType("TOKEN");

    assertThrows(PaymentProcessingException.class,
        () -> paymentService.createPayment(request, 200L));
    verify(paymentRepository, never()).save(any(Payment.class));
}

@Test
@DisplayName("Should reject payment with negative amount")
void testCreatePayment_NegativeAmount_ThrowsException() {
    PaymentRequest request = new PaymentRequest();
    request.setBookingId(100L);
    request.setAmount(new BigDecimal("-5000.00")); // Negative amount
    request.setPaymentMethod(PaymentMethod.CARD);
    request.setPaymentType("TOKEN");

    assertThrows(PaymentProcessingException.class,
        () -> paymentService.createPayment(request, 200L));
    verify(paymentRepository, never()).save(any(Payment.class));
}
```

B. Idempotency & Duplicate Payment Prevention

If a user accidentally clicks "Pay Now" twice or network packets duplicate, the system checks existing completed payments for that booking and rejects duplicate token charges:

```
@Test
@DisplayName("Should throw PaymentProcessingException on duplicate token payment for same booking")
void testCreatePayment_DuplicateToken_ThrowsException() {
    PaymentRequest request = new PaymentRequest();
    request.setBookingId(100L);
    request.setAmount(new BigDecimal("5000.00"));
    request.setPaymentType("TOKEN");

    // Mocking existing completed payment
    when(paymentRepository.findByBookingId(100L)).thenReturn(List.of(sampleTokenPayment));

    assertThrows(PaymentProcessingException.class, () -> paymentService.createPayment(request, 200L));
    verify(paymentRepository, never()).save(any(Payment.class));
}
```

6. TOP 8 UNIT TESTING VIVA QUESTIONS & MODEL ANSWERS

#	Examiner Question	Your Bulletproof Model Answer
1	What is the difference between @Mock and @InjectMocks?	"@Mock creates a fake dummy instance of a dependency (like UserRepository). @InjectMocks creates the real instance of the service class under test and injects those mock objects into its fields or constructor."
2	What is Mockito Stubbing?	"Stubbing is programming a mock object's behavior using when().thenReturn(). It tells the mock: 'When method X is called with argument Y, return fake answer Z without executing real database queries.'"

#	Examiner Question	Your Bulletproof Model Answer
3	Why do you verify() with never() in negative tests?	<i>"In negative exception tests, verifying never().save() proves data integrity: if an error or duplicate user occurs, the system halted execution immediately and never attempted to write corrupted data to MySQL."</i>
4	How does MockMvc test controllers without Tomcat?	<i>"MockMvc uses Spring's MockServletContext to build a mock HTTP environment. It serializes JSON payloads, invokes controller mapping annotations, and verifies HTTP status codes and JSON responses in memory."</i>
5	Why use ParameterizedTest in JUnit 5?	<i>"@ParameterizedTest with @ValueSource allows executing the same test logic across multiple input variations (like 10 different invalid phone numbers or email strings) without duplicating code."</i>
6	How do you test the AI Rent Estimator?	<i>"In PropertyServiceTest, we feed a complete RentEstimationRequest and assert that estimatedRent is positive, confidenceScore >= 90, and minRent < estimatedRent < maxRent to guarantee regression stability."</i>
7	How do you test Government KYC without UIDAI server access?	<i>"We test the KYC engine's algorithmic validation: regex pattern conformity for Aadhaar and PAN, data masking functions (XXXX-XXXX-1098), and credit scoring calculations in complete isolation."</i>
8	What is JaCoCo?	<i>"JaCoCo (Java Code Coverage) is a bytecode instrumentation tool configured in our pom.xml that analyzes test executions and generates HTML reports showing line, branch, and instruction coverage."</i>